

Data Structures

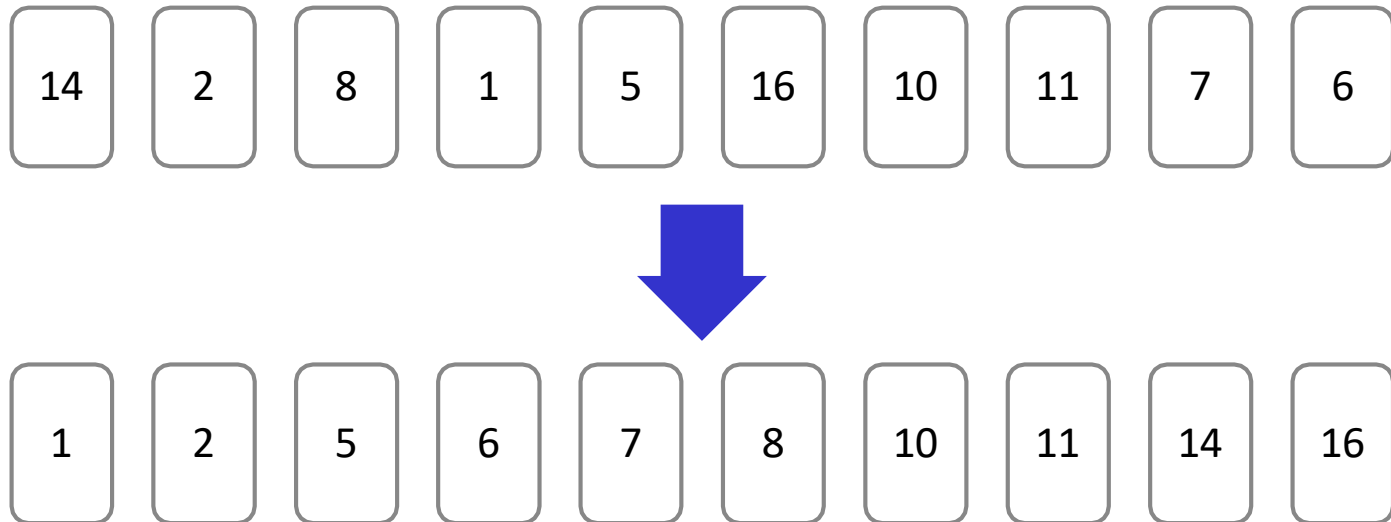
Lecture 14: Merge Sort

Soobin Um

May 4, 2026

Recap: Sorting (정렬)

- 리스트의 아이템들을 일정 기준(예: 오름차순, 내림차순)에 따라 **순서화하는 작업**
- **정렬 알고리즘의 종류**
 - **Basic:** Bubble sort, Selection sort, Insertion sort
 - 직관적이지만 다소 비효율적
 - **Advanced:** Quick sort, Merge sort, Heap sort
 - 분할정복 전략과 효율적인 자료 구조를 활용하여 정렬 속도를 크게 개선



Recap: Basic Sorting Algorithms

- **Bubble sort:** “두 개씩 보고 큰 것을 오른쪽으로 보내자”
- **Selection sort:** “가장 작은 것을 선택하여 가장 왼쪽에 놓자”
- **Insertion sort:** “일단 하나를 잡고, 적절한 위치에 끼워 넣자”

알고리즘	전략	시간 복잡도 (비교 횟수 기준)	특징
Bubble Sort	큰 값을 뒤로 이동	$O(n^2)$	Swap 연산이 많음
Selection Sort	작은 값을 앞으로 이동	$O(n^2)$	Swap 연산이 적음
Insertion Sort	정렬 구간에 삽입	$O(n^2)$	거의 정렬되어 있는 리스트에서 매우 효과적

Merge Sort

Divide and Conquer (분할 정복 전략)

Divide and Conquer (분할 정복 전략)

1. 큰 문제를 여러 개의 작은 하위 문제로 나눔 (divide)

Divide and Conquer (분할 정복 전략)

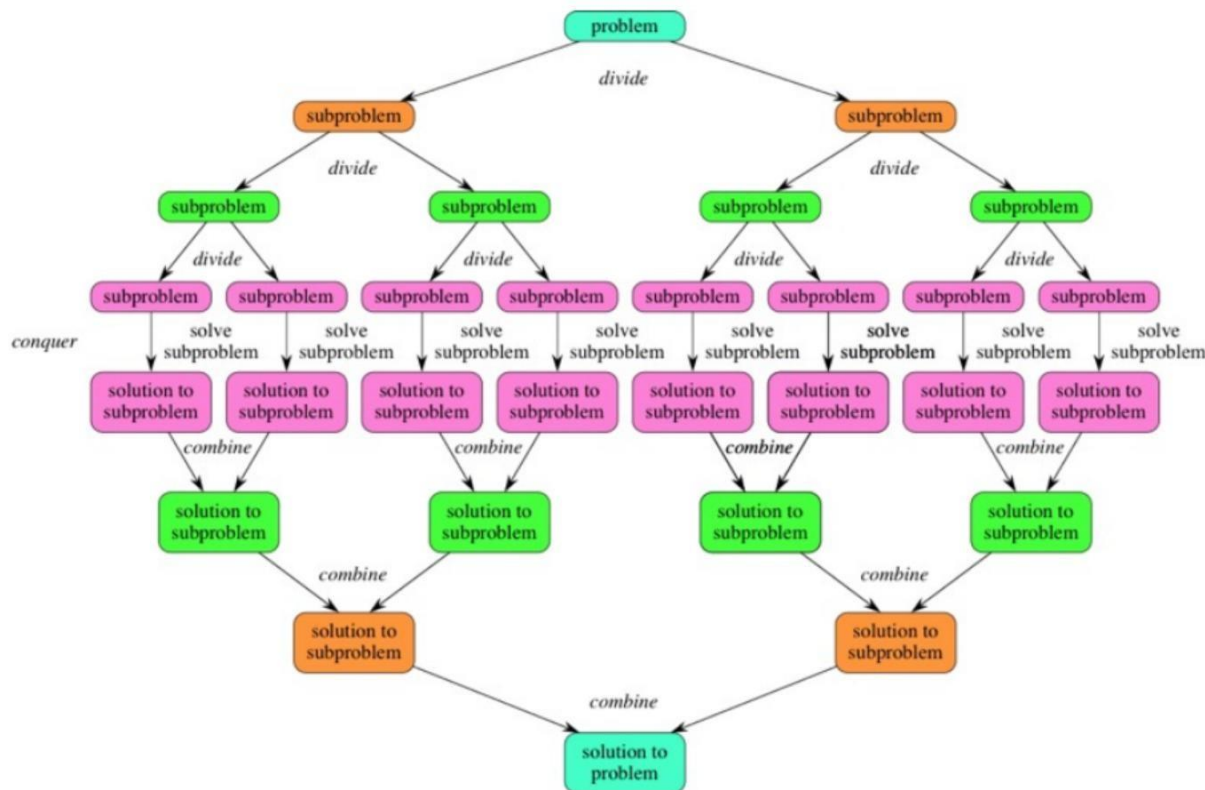
1. 큰 문제를 여러 개의 작은 하위 문제로 나눔 (divide)
2. 각 하위 문제를 개별적으로 해결 (conquer)

Divide and Conquer (분할 정복 전략)

1. 큰 문제를 여러 개의 작은 하위 문제로 나눔 (divide)
2. 각 하위 문제를 개별적으로 해결 (conquer)
3. 하위 문제들의 해결 결과를 결합하여 (merge) 원래 문제를 해결

Divide and Conquer (분할 정복 전략)

1. 큰 문제를 여러 개의 작은 하위 문제로 나눔 (divide)
2. 각 하위 문제를 개별적으로 해결 (conquer)
3. 하위 문제들의 해결 결과를 결합하여 (merge) 원래 문제를 해결



Merge Sort (병합 정렬)

1. Divide:
2. Conquer:
3. Merge:

Merge Sort (병합 정렬)

1. Divide: 입력 배열을 두 개의 절반 크기 하위 배열로 나눔
2. Conquer:
3. Merge:

Merge Sort (병합 정렬)

1. Divide: 입력 배열을 두 개의 절반 크기 하위 배열로 나눔
2. Conquer: 각 하위 배열을 병합 정렬을 이용해 재귀적으로 정렬
3. Merge:

Merge Sort (병합 정렬)

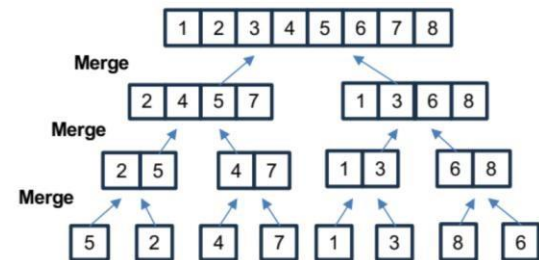
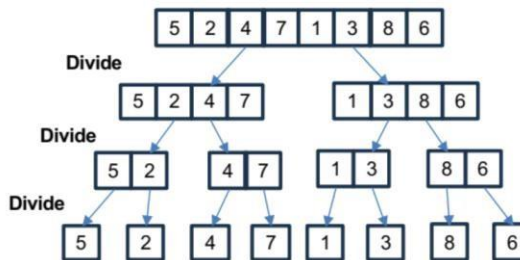
1. Divide: 입력 배열을 두 개의 절반 크기 하위 배열로 나눔
2. Conquer: 각 하위 배열을 병합 정렬을 이용해 재귀적으로 정렬
3. Merge: 정렬된 두 하위 배열을 병합하여 최종적으로 정렬된 리스트를 만듦

Merge Sort (병합 정렬)

1. Divide: 입력 배열을 두 개의 절반 크기 하위 배열로 나눔
2. Conquer: 각 하위 배열을 병합 정렬을 이용해 재귀적으로 정렬
3. Merge: 정렬된 두 하위 배열을 병합하여 최종적으로 정렬된 리스트를 만듦

MERGE-SORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )
```

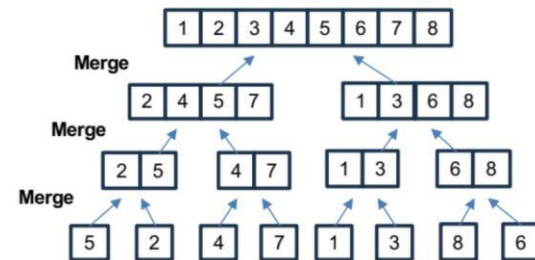
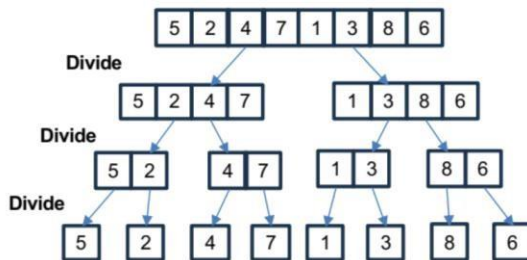


Merge Sort (병합 정렬)

1. Divide: 입력 배열을 두 개의 절반 크기 하위 배열로 나눔
2. Conquer: 각 하위 배열을 병합 정렬을 이용해 재귀적으로 정렬
3. Merge: 정렬된 두 하위 배열을 병합하여 최종적으로 정렬된 리스트를 만듦

MERGE-SORT(A, p, r)

```
1  if  $p < r$ 
2     $q = \lfloor (p + r) / 2 \rfloor$ 
3    MERGE-SORT( $A, p, q$ )
4    MERGE-SORT( $A, q + 1, r$ )
5    MERGE( $A, p, q, r$ )
```



Question: “두 개의 sorted subarrays를 어떻게 병합하나?”

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

--	--	--	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2									
---	--	--	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2									
---	--	--	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3								
---	---	--	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3								
---	---	--	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4							
---	---	---	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4							
---	---	---	--	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5						
---	---	---	---	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5						
---	---	---	---	--	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6					
---	---	---	---	---	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6					
---	---	---	---	---	--	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7				
---	---	---	---	---	---	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7				
---	---	---	---	---	---	--	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7	8			
---	---	---	---	---	---	---	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7	8			
---	---	---	---	---	---	---	--	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7	8	9		
---	---	---	---	---	---	---	---	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7	8	9		
---	---	---	---	---	---	---	---	--	--

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7	8	9	12	13
---	---	---	---	---	---	---	---	----	----

Merging Two Sorted Arrays

3	4	7	12	13
---	---	---	----	----

2	5	6	8	9
---	---	---	---	---

2	3	4	5	6	7	8	9	12	13
---	---	---	---	---	---	---	---	----	----

두 개의 sorted subarrays를 병합하는 연산은 $O(n)$ ($n=10$)의 시간 복잡도를 지님

Merge Sort

- 하나의 리스트가 병합 정렬을 통해 정렬되는 전체 과정
 - Divide – Conquer – Merge

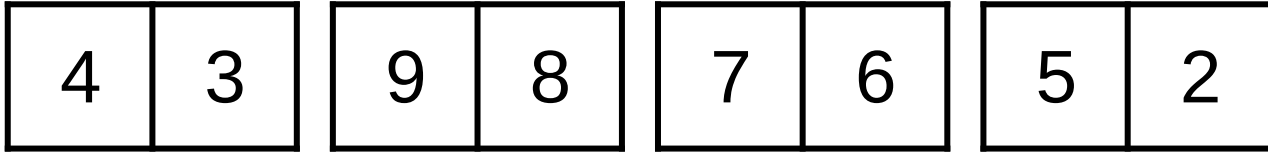
4	3	9	8	7	6	5	2
---	---	---	---	---	---	---	---

Merge Sort: Divide

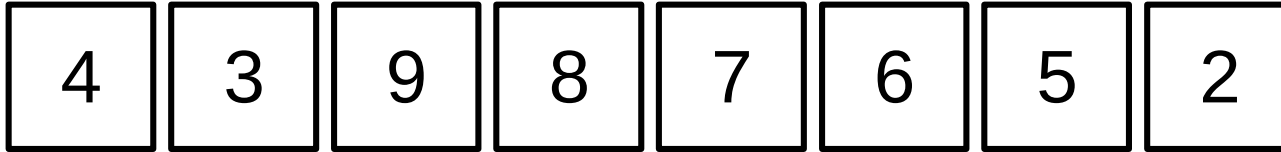
4	3	9	8
---	---	---	---

7	6	5	2
---	---	---	---

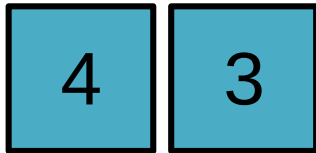
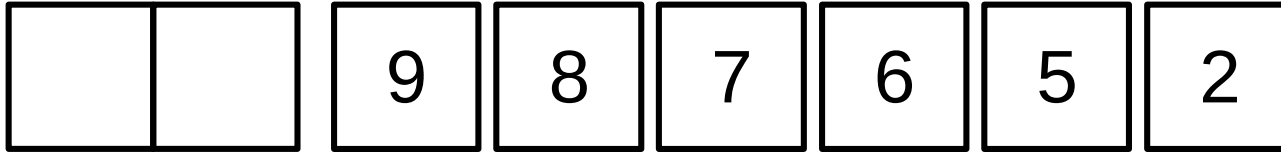
Merge Sort: Divide



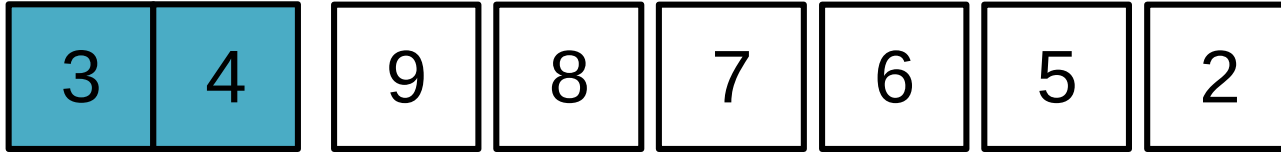
Merge Sort: Divide



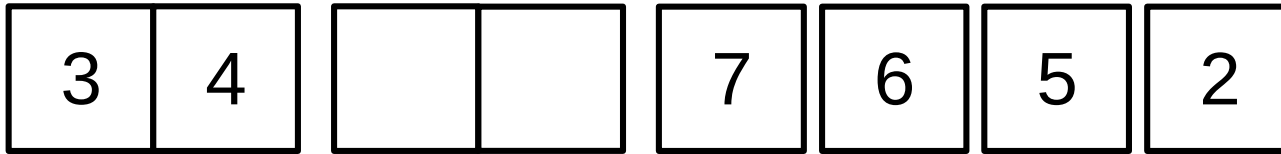
Merge Sort: Conquer



Merge Sort: Merge



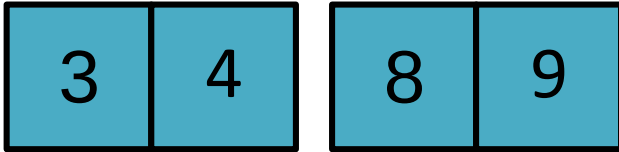
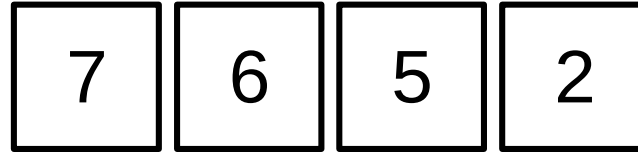
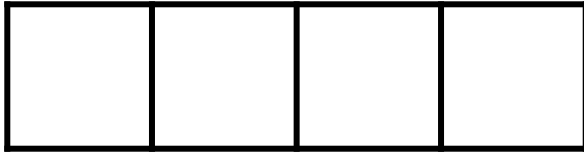
Merge Sort: Conquer



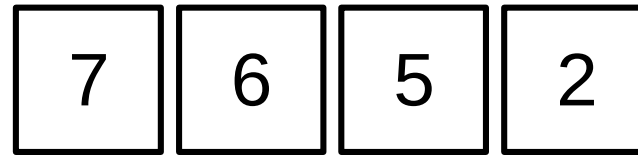
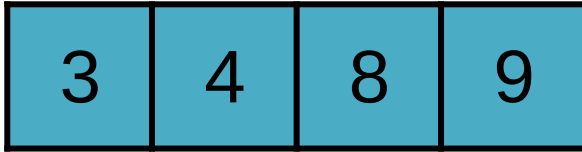
Merge Sort: Merge



Merge Sort: Conquer



Merge Sort: Merge



Merge Sort: Conquer and Merge



Merge Sort: Conquer and Merge

3	4	8	9
---	---	---	---

6	7
---	---

2	5
---	---

Merge Sort: Conquer

3	4	8	9
---	---	---	---

--	--	--	--

6	7
---	---

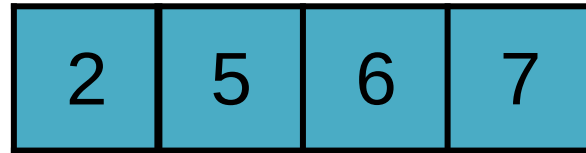
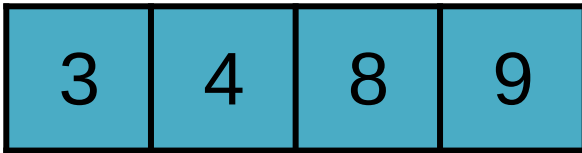
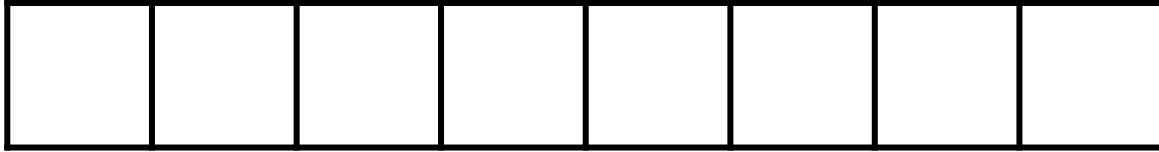
2	5
---	---

Merge Sort: Merge

3	4	8	9
---	---	---	---

2	5	6	7
---	---	---	---

Merge Sort: Conquer



Merge Sort: Merge (완성)

2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---

구현: mergeSort()

- 리스트 길이가 1 이하일 경우 리스트를 그대로 반환, 그렇지 않을 경우 리스트를 반으로 나눈 뒤 재귀 호출

```
List<Integer> mergeSort(List<Integer> inList) {  
    /* 리스트의 길이가 0 또는 1이면 그대로 반환 */  
    if (inList.size() <= 1)  
        return inList;  
  
    /* 리스트를 반으로 분할 */  
    int mid = inList.size() / 2;  
    List<Integer> left = new ArrayList<>(inList.subList(0, mid));  
    List<Integer> right = new ArrayList<>(inList.subList(mid, inList.size()));  
  
    /* 재귀 호출 후 merge */  
    return merge(mergeSort(left), mergeSort(right));  
}
```

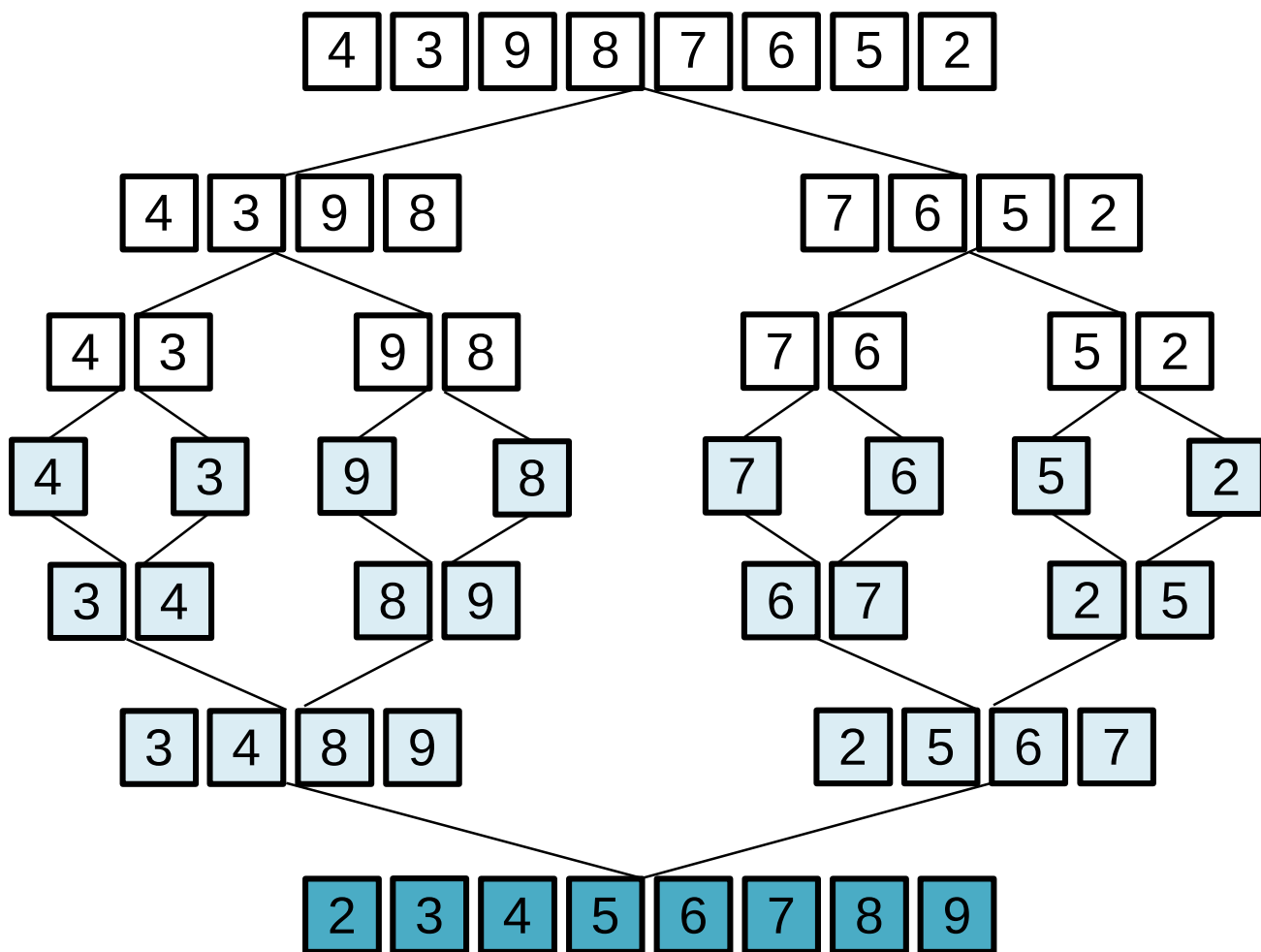
구현: merge()

- 정렬된 두 리스트를 비교하며 하나의 정렬된 리스트로 병합

```
List<Integer> merge(List<Integer> left, List<Integer> right) {  
    List<Integer> result = new ArrayList<>();  
    int i = 0, j = 0;  
  
    /* 양쪽 리스트에서 작은 값부터 result에 추가 */  
    while (i < left.size() && j < right.size()) {  
        ... / Homework #2  
    }  
  
    /* 남아있는 원소들을 result에 추가 */  
    while (i < left.size()) {  
        ... / Homework #2  
    }  
    while (j < right.size()) {  
        ... / Homework #2  
    }  
  
    return result;  
}
```

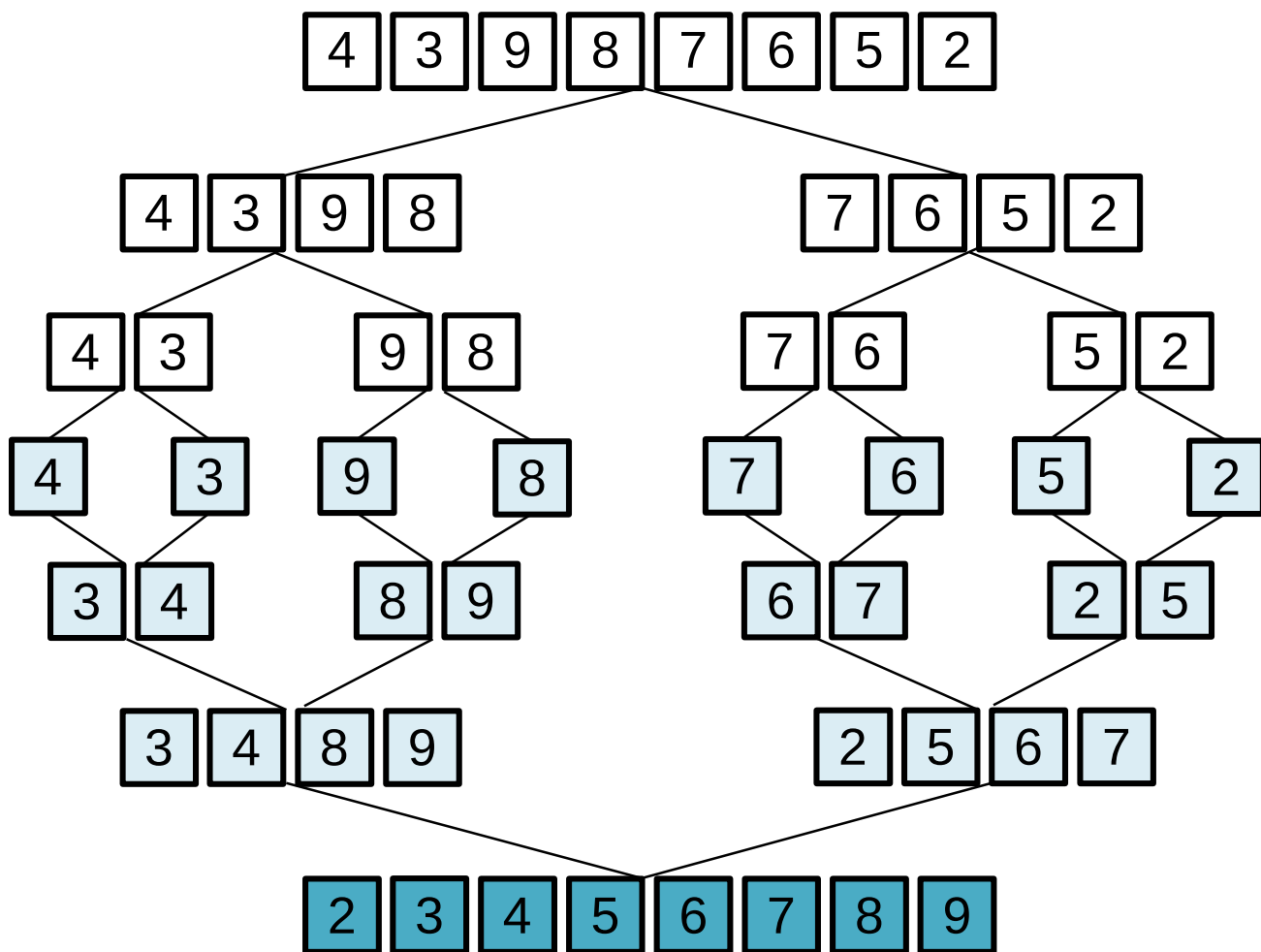
시간 복잡도

- 재귀의 깊이는 $\log n$ 이며, 각 레벨에서 총 n 개의 원소가 병합 과정에 1번 씩 참여



시간 복잡도

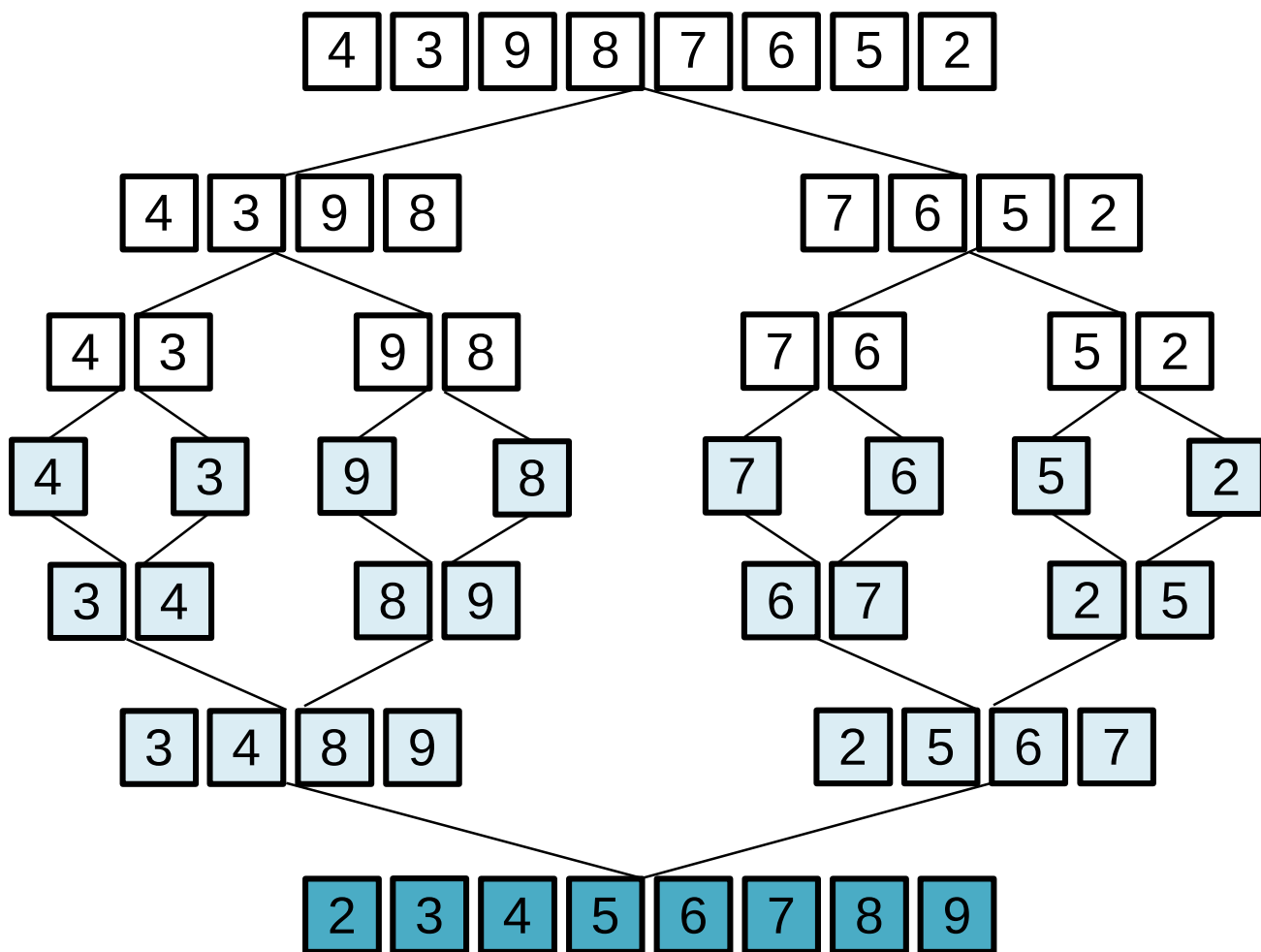
- 재귀의 깊이는 $\log n$ 이며, 각 레벨에서 총 n 개의 원소가 병합 과정에 1번 씩 참여



깊이는 3 ($=\log n$)
각 레벨에서 8 ($=n$)개의
원소가 참여

시간 복잡도

- 재귀의 깊이는 $\log n$ 이며, 각 레벨에서 총 n 개의 원소가 병합 과정에 1번 씩 참여



깊이는 3 ($=\log n$)
각 레벨에서 8 ($=n$)개의
원소가 참여

병합 정렬의 시간 복잡도는 $O(n \log n)$

Look ahead

Quick Sort